AP CSA: Common Compile Errors (CE)

11 Most Frequent Compiler Errors: Definitions, Causes, Broken vs. Fixed Examples

01. cannot find symbol

Definition: The compiler encounters an identifier (variable, method, or class) that has not been declared or is out of scope.

'L BROKEN

```
for (int i=0; i<5; i++) {
  int secret = 42;
}
System.out.println(secret); // CE: cannot find symbol
```

' FIXED SOLUTION

```
int secret = 0; // Declare in outer scope
for (int i=0; i<5; i++) {
  secret = 42;
}
System.out.println(secret); // OK: prints 42
```

02. incompatible types: possible lossy conversion from double to int

Definition: Attempting to store a floating-point number (double) into an int variable without an explicit cast.

'L BROKEN

```
int count = 5.8;     // CE: lossy conversion
int half = 10 / 4.0; // CE: 4.0 makes result double
```

' FIXED SOLUTION

```
int count = (int) 5.8;       // Truncates to 5
int half = (int) (10 / 4.0); // Truncates to 2
```

03. variable <var> might not have been initialized

Definition: Reading or printing a local method variable before Java can guarantee an initial value on every branch.

'L BROKEN

```
int grade; int score = 75;
if (score >= 90) grade = 1;
System.out.println(grade); // CE: uninitialized
```

' FIXED SOLUTION

```
int grade = 0; // Safely initialized
int score = 75;
if (score >= 90) grade = 1;
System.out.println(grade); // OK
```

04. missing return statement

Definition: A non-void method promised to return a value, but there is a path where execution finishes without returning.

'L BROKEN

```
public boolean isPositive(int n) {
  if (n > 0) return true;
  // CE: missing return statement if n <= 0
}
```

' FIXED SOLUTION

```
public boolean isPositive(int n) {
  return n > 0; // Concise and guaranteed
}
```

05. unreachable code / unreachable statement

Definition: Statements placed directly after a line that unconditionally exits the method or loop (like return or break).

'L BROKEN

```
public int getVal() {
  return 10;
  System.out.println("Done"); // CE: unreachable
}
```

' FIXED SOLUTION

```
public int getVal() {
  System.out.println("Done");
  return 10; // Placed at exit
}
```

06. non-static method cannot be referenced from a static context

Definition: Calling an instance method directly from static main without creating an object instance first.

'L BROKEN

```
public class Game {
  public void play() { ... }
  public static void main(String[] args) {
    play(); // CE: non-static method in static context
  }
}
```

' FIXED SOLUTION

```
public static void main(String[] args) {
  Game g = new Game();
  g.play(); // OK: called on instance g
}
```

07. array required, but ArrayList found

Definition: Using array brackets [ ] on an ArrayList, or calling .get() / .size() on a basic 1D/2D array.

'L BROKEN

```
ArrayList<String> list = new ArrayList<>();
list.add("Java");
System.out.println(list[0]); // CE: array required
```

' FIXED SOLUTION

```
System.out.println(list.get(0)); // OK: prints Java
System.out.println(list.size());  // OK: prints 1
```

08. constructor in class cannot be applied to given types

Definition: Calling new ClassName(...) or super(...) with parameters that do not match any constructor.

'L BROKEN

```
class Student { public Student(String name) {} }
Student s = new Student(); // CE: constructor mismatch
```

' FIXED SOLUTION

```
Student s = new Student("Franky"); // OK: matches
parameter
```

09. bad operand types for binary operator (<, >, ==)

Definition: Applying arithmetic or relational operators to objects (like Strings) that do not support them.

'L BROKEN

```
String s1 = "apple"; String s2 = "banana";
if (s1 < s2) { ... } // CE: bad operand types
```

' FIXED SOLUTION

```
if (s1.compareTo(s2) < 0) {
  System.out.println(s1 + " comes before " + s2);
}
```

10. 'void' type not allowed here

Definition: Trying to print, assign, or pass the return value of a method that returns void (no value).

'L BROKEN

```
ArrayList<String> list = new ArrayList<>();
System.out.println(list.clear()); // CE: void not allowed
```

' FIXED SOLUTION

```
list.clear();                    // Separate statement
System.out.println("Cleared!"); // OK
```

11. class, interface, or enum expected

Definition: An unexpected closing curly brace } closed the class prematurely, leaving code floating outside.

'L BROKEN

```
public class Demo {
  public static void main(String[] args) {}
} // Extra brace ends class!
  public void helper() {} // CE: class expected
```

' FIXED SOLUTION

```
public class Demo {
  public static void main(String[] args) {}
  public void helper() {} // Properly enclosed
}
```